UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

S6

Algoritmos genéticos: ruteo de reparto

Actividad Parcial 2 · Fundamentos de Inteligencia Artificial

Tema 2.6 · Proyecto final en equipo con DEAP

Equipo	TurboRucos
Integrantes	Denise Estrada · Anahi Fuentes · Natanael Rosales · Rogerio Aguilar
Grupo / cuatrimestre	171 · 7.º
Docente	Mtro. Javier García Lara
Tipo de entrega	Equipo · 50 pts
Fecha de entrega	26 de octubre de 2026
Referencias del tema	Wirsansky (2020), caps. 3–5
Repositorio GitHub	https://github.com/RogerNika/Rogerio-Aguilar-Bello-FIA/tree/main/FIA_G171_Aguilar_S3

Plus de esta actividad: estudio de ablación de operadores genéticos y animación de la evolución de la mejor ruta.

Capítulo 1. Planteamiento del problema

1.1 Equipo y roles

Rol	Responsabilidad	Responsable
Coordinación	Plan de trabajo, entregas y revisión final	[Integrante 1]
Modelado	Cromosoma, fitness y datos del caso	[Integrante 2]
Implementación	Código con DEAP y experimentos	[Integrante 3]
Análisis y gráficas	Convergencia, comparaciones y plus	Rogerio [Apellidos]
Documentación y GitHub	README, informe y repositorio	[Todos]

1.2 Caso empresarial

Una distribuidora regional (caso **ilustrativo**) reparte producto desde Amecameca a 13 comunidades del oriente del Estado de México y Morelos y debe regresar al punto de partida. Decidir el orden de visita que minimice los kilómetros recorridos reduce combustible, tiempo y emisiones.

1.3 Problema y datos

Es un problema del agente viajero (TSP) con 14 ciudades. El espacio de búsqueda tiene $(14 - 1)!/2 = 3{,}113{,}510{,}400$ rutas distintas, demasiadas para revisarlas una por una a mano. Las distancias se **estiman** como 1.3 × la distancia en línea recta entre coordenadas aproximadas de las cabeceras (modelo didáctico); con datos reales basta reemplazar la matriz de distancias.

Capítulo 2. Objetivos

2.1 Objetivo general

Resolver el problema de ruteo de reparto mediante un algoritmo genético implementado con DEAP en Python para obtener rutas cercanas al óptimo con una fracción del espacio de búsqueda evaluada.

2.2 Objetivos específicos

- **Definir** el cromosoma, el fitness y los operadores genéticos **adaptándolos** a un problema de permutaciones **para** representar rutas válidas.
- **Ejecutar** el algoritmo con una población de 100 individuos durante 40 generaciones y 10 semillas **para** evaluar su calidad y estabilidad.
- **Comparar** el resultado con el óptimo exacto (Held-Karp) y con el vecino más cercano **para** medir la brecha de calidad.
- **Documentar** un instructivo de replicación paso a paso **mediante** un README y un repositorio en GitHub **para** que cualquier persona reproduzca los resultados.

Capítulo 3. Fundamentos

Un **algoritmo genético** (AG) mantiene una población de soluciones candidatas (individuos) y las mejora por generaciones imitando la selección natural: los individuos mejor evaluados (fitness) se seleccionan, se cruzan para combinar sus características y se mutan para introducir variedad (Goldberg, 1989; Wirsansky, 2020).

Cromosoma	Permutación de las 14 ciudades: el orden de visita (ruta cerrada).
Fitness	Longitud total de la ruta en km; se minimiza.
Población	100 individuos aleatorios; 40 generaciones.
Selección	Torneo de tamaño 3.
Cruce	Ordenado (OX), con probabilidad 0.8: conserva un tramo de un padre y completa con el orden del otro, sin repetir ciudades.
Mutación	Intercambio de índices (`mutShuffleIndexes`, indpb = 0.05) con probabilidad 0.2.
Elitismo	Los 2 mejores pasan sin cambios a la siguiente generación.

Se usó la biblioteca DEAP (Fortin et al., 2012). Como referencias se calculó el óptimo exacto con programación dinámica de Held y Karp (1962), viable con 14 ciudades, y la heurística del vecino más cercano.

Capítulo 4. Metodología y procedimiento

- Construir la matriz de distancias entre las 14 ciudades.
- Calcular las referencias: óptimo exacto (Held-Karp), vecino más cercano y ruta aleatoria promedio (2,000 muestras).
- Ejecutar el AG con 10 semillas, registrando el mejor y el promedio de fitness por generación.
- Comparar costo final, brecha frente al óptimo, generaciones para llegar a ≤ 1 % del óptimo y evaluaciones.
- Realizar un estudio de ablación de operadores (plus) con 12 semillas por combinación.
- Documentar el instructivo de replicación y subirlo a GitHub.

Capítulo 5. Desarrollo: implementación

- **Inicialización:** `random.sample(range(14), 14)` genera permutaciones válidas.
- **Evaluación:** suma de distancias entre ciudades consecutivas, incluyendo el regreso.
- **Ciclo evolutivo:** `varAnd` aplica cruce y mutación sobre la población seleccionada; se reinsertan los 2 mejores (elitismo).
- **Validez:** OX, PMX y las mutaciones conservan permutaciones, por lo que no se requieren reparaciones.

Capítulo 6. Resultados

Método	Longitud (km)	Brecha vs. óptimo
Ruta aleatoria (media)	543.6	+103 %
Vecino más cercano	284.8	+6.2 %
AG: media de 10 corridas	272.5 ± 4.0	+1.56 %
AG: mejor corrida	268.3	+0.00 %
AG: peor corrida	279.2	+4.05 %
Óptimo exacto (Held-Karp)	268.3	0 %

Mejor ruta del AG: Ecatzingo → Atlautla → Ozumba → Amecameca → Tlalmanalco → Cocotitlán → Ixtapaluca → Texcoco → Ciudad de México → Nezahualcóyotl → Chalco → Tenango del Aire → Juchitepec → Cuautla.

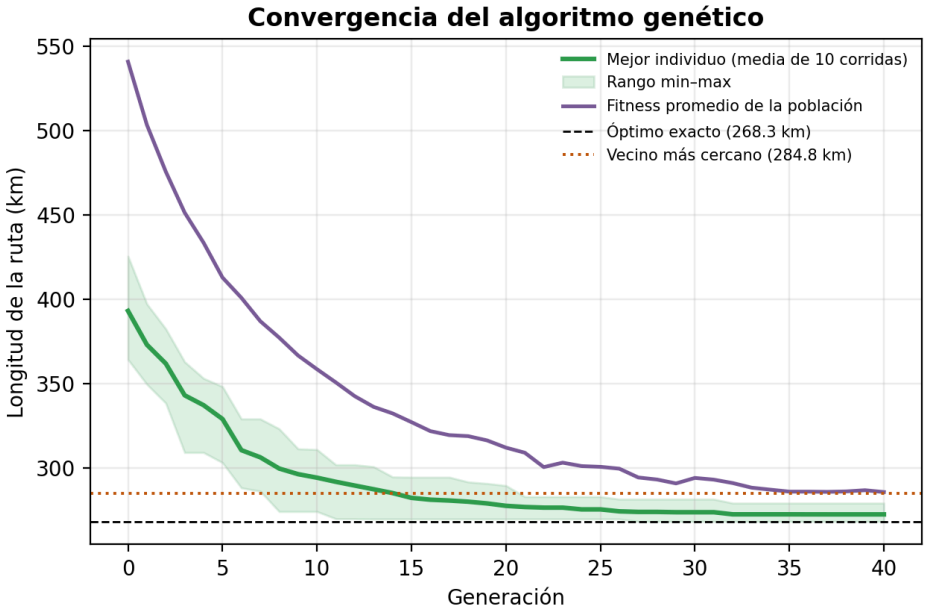


Figura 1. Convergencia: mejor individuo (media y rango de 10 corridas) y fitness promedio de la población.

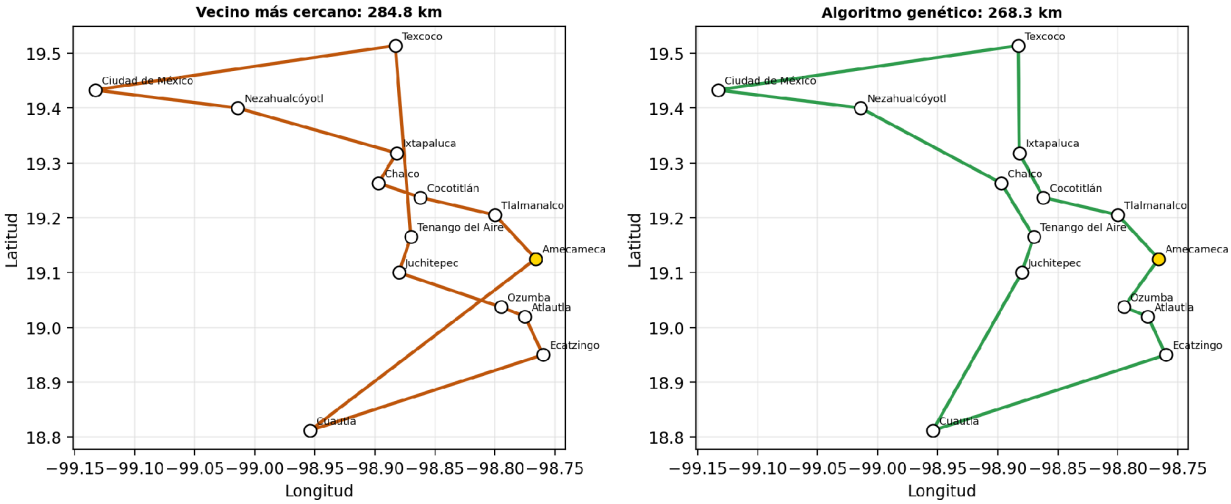


Figura 2. Mejor ruta del algoritmo genético frente a la del vecino más cercano.

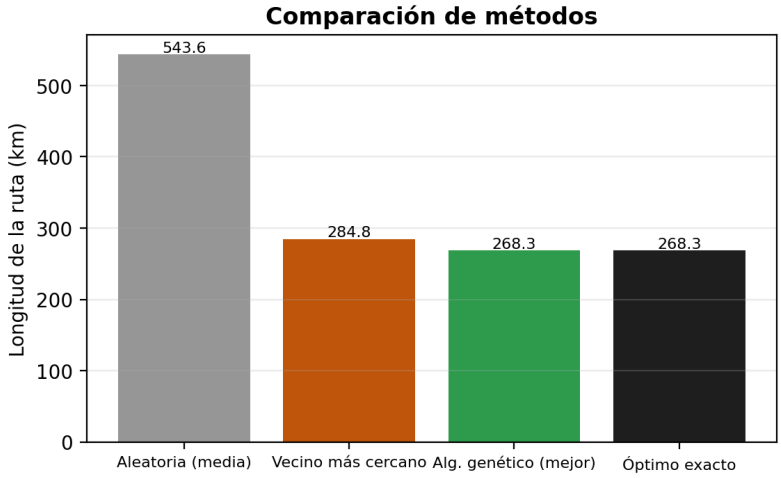


Figura 3. Comparación de longitud de ruta por método.

Capítulo 7. Código Python

Archivo `fia_s6_algoritmo_genetico.py` (requiere deap, numpy, matplotlib, pillow y `datos_edomex.py`).

```
W
```

Capítulo 8. Análisis y discusión

8.1 Calidad de la solución

El AG mejoró al vecino más cercano en promedio (272.5 km frente a 284.8 km, 4.3 % menos) y encontró el óptimo exacto en 10% de las corridas (268.33 km). La brecha media fue de 1.56 %. Para lograrlo evaluó 4,020 rutas, es decir, 0.00013 % del espacio de búsqueda.

8.2 Convergencia

La mayor parte de la mejora ocurre en las primeras generaciones; después la curva se aplana. 6 de 10 corridas llegaron a menos de 1 % del óptimo entre las generaciones 11 y 32, y las otras 4 no lo lograron en 40 generaciones, por lo que con más generaciones o más población la tasa de éxito aumentaría.

8.3 Efecto de los operadores

La configuración base (OX con mutación de intercambio de índices) fue la más débil entre las combinaciones con OX (brecha de 1.97 %). En el estudio de ablación, la mejor combinación fue OX con inversión (brecha de 0.19 %) y la peor PMX con intercambio de índices (3.76 %). Las mutaciones por inversión y swap funcionan mejor porque hacen cambios más pequeños y estructurados en la ruta, y OX conserva mejor el orden relativo que PMX en este problema. Se recomienda usar OX con inversión como configuración final.

8.4 Limitaciones

- Las distancias son estimadas y las demandas, capacidades y ventanas de tiempo no se modelaron.
- Con 14 ciudades el óptimo exacto es calculable; con cientos de paradas el AG sería necesario y la brecha no podría medirse igual.
- Los resultados dependen de la semilla; por eso se reportan 10 corridas.

Capítulo 9. Conclusiones

- El AG encontró rutas a 1.6 % del óptimo en promedio evaluando una fracción mínima del espacio, y superó al vecino más cercano.
- Los operadores importan: cambiar la mutación redujo la brecha de aproximadamente 2 % a menos de 0.3 % en el estudio de ablación.
- Disponer de un óptimo exacto en un caso pequeño permite medir con honestidad la calidad del AG.
- El instructivo y el repositorio hacen reproducible el trabajo para el equipo y para terceros.

Capítulo 10. Instructivo de replicación paso a paso

- **Clonar el repositorio** y entrar a la carpeta `S6/`.
- **Crear un entorno** (opcional): `python -m venv .venv` y activarlo.
- **Instalar dependencias:** `pip install deap numpy matplotlib pillow`.
- **Ejecutar:** `python fia_s6_algoritmo_genetico.py` (tarda menos de un minuto).
- **Revisar los resultados:** `salida_s6.txt`, `resultados_s6.json`, las figuras `s6_*.png` y `evolucion_ruta.gif`.
- **Cambiar el caso:** editar la lista `CIUDADES` (y sus coordenadas en `datos_edomex.py`) o reemplazar la matriz D con distancias reales.
- **Cambiar parámetros:** argumentos de `correr_ga` (`n_pob`, `n_gen`, `cxpb`, `mutpb`, `cruce`, `mutacion`).

Capítulo 11. Plus: ablación de operadores y evolución de la ruta

Ablación. Se compararon dos cruces (OX y PMX) con tres mutaciones (intercambio de índices, inversión y swap), con 12 corridas por combinación y semillas distintas a las del experimento principal.

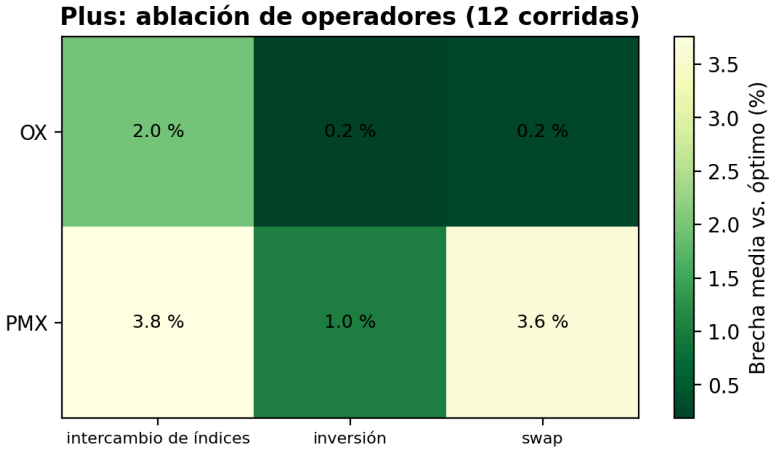


Figura 4. Brecha media frente al óptimo (%) por combinación de operadores.

Evolución de la ruta. El archivo `evolucion_ruta.gif` muestra cómo la mejor ruta pasa de un recorrido desordenado a uno casi óptimo; la figura reúne cuatro cuadros.

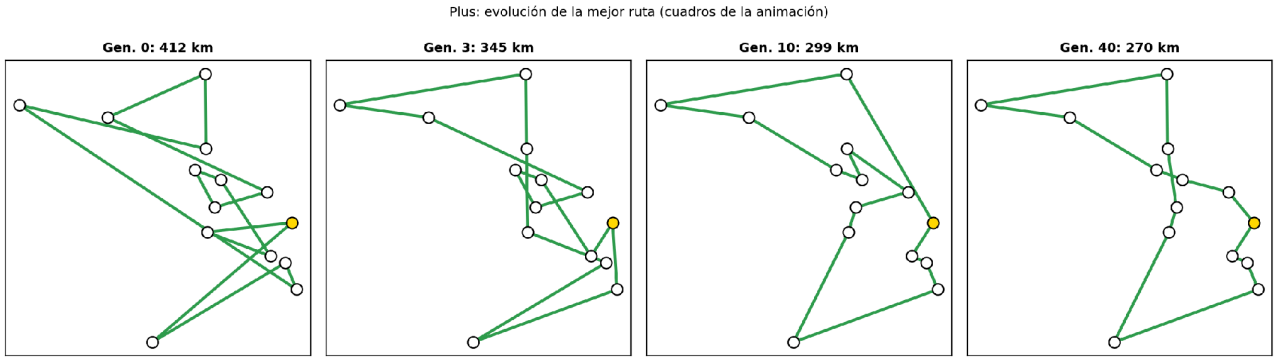


Figura 5. Mejor ruta en las generaciones 0, 3, 10 y 40 de una corrida.

Capítulo 12. Referencias (APA 7.ª ed.)

Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research, 13*, 2171–2175.

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization and machine learning*. Addison-Wesley.

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics, 10*(1), 196–210. https://doi.org/10.1137/0110015

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python*. Packt.

Anexo A. Lista de cotejo (S6)

#	Criterio de evaluación	S6
1	Portada, tema, sección y datos del problema completos en el PDF	✓
2	Planteamiento, formulación y procedimiento claramente explicados	✓
3	Código Python funcional y comentado, probado con datos de entrada	✓
4	Solución enmarcada, con gráficos, tablas o diagramas de resultados	✓
5	Repositorio GitHub con carpeta del alumno, README y PDFs	Pendiente: subir a GitHub
6	Declaración de IA utilizada, referencias APA 7 y conclusiones reflexivas	✓

Declaración de uso de IA

Herramienta: Claude (Anthropic), asistente conversacional.
Prompt: Diseña y da claridad a la informacion, modifica donde sea necesario y revisa el codigo para verificar funcionalidad
Uso: apoyo para plantear el caso, escribir el código con DEAP, ejecutar los experimentos, redactar el instructivo y el borrador.
Verificación: el código se ejecutó y su salida se incluye en el capítulo 7; el óptimo se verificó con programación dinámica exacta; las cifras del texto se calcularon desde el archivo de resultados. Revisión personal por el alumno: [fecha].